Gym Space Monitoring System

Software Architecture Design

SAD Version 1.0

Team T08

21 April 2026

Nathan McConnell (Manager)

Ashton Langen

Kevin Rouzaud

Mckenzie Johnston

Julia Marsh

CS 460 Software Engineering

Table Of Contents

1 Introduction **2**

2 Design Overview **2**

2.1 Design Explanation 3

3 Component Specification **6**

3.1 Alert Manager 6

3.2 Door Scanner Controller 7

3.3 Employee Dashboard 8

3.4 Web Service 8

3.5 AI Hazard Analyzer 9

3.6 Sound Monitor 10

3.7 Air Quality/Temp Monitor 11

3.8 Machine Monitor 11

4 Use Cases **12**

5 Design Constraints **18**

5.1 Maintainability 18

5.2 AI and Data Processing Constraints 18

5.3 Environmental Constraints 19

5.4 Privacy Constraints 19

5.5 Equipment and Integration Constraints 19

5.6 System Reliability and Recovery 19

6 Definition of Terms **20**

7 Demonstration Semantics **21**

8 References **23**

1 Introduction

The purpose of this document is to describe the requirements that regulate the design of the Gym Space Monitoring System (GSMS) along with software implementation details. Throughout this document, technical blueprints, structural frameworks, and implementation strategies underlying the GSMS will be communicated in comprehensive detail. The software for the GSMS will be in charge of managing the continuous ingestion of separate data streams, stemming from AI-based computer vision feeds, environmental sensors, and electronic machine usage signals, all asynchronously. Furthermore, it is responsible for routing the correct system state through alerts sent to connected software components as well as logs sent to the data store. The GSMS software is also in charge of managing and parsing physical hardware responses, such as triggering the Bulky Buzzer when localized noise thresholds are exceeded, and managing and parsing through API calls to the AI computer vision camera feeds. Due to the concurrent responsibilities that the GSMS software is tasked with, it is paramount that special care is put into the design of the system, ensuring that the system can respond to any use case in a fast and efficient manner, all while fulfilling our predefined goals and constraints.

This document will present, in detail, our design for the GSMS software in six additional sections. *Section 2* will go over the design overview, detailing the overall design architecture, with the visual aid of a block diagram, along with an explanation of this diagram. *Section 3* will go through the various important function calls and their descriptions, each pivotal to understanding how that component interacts with the system on a software level. *Section 4* will then go through the use cases of the GSMS, showcasing how these cases align with our architecture, and specify which components contribute to each use case. *Section 5* will list the design constraints which need to be kept in mind during our designing process. *Section 6* will go through the specialized terms and definitions which were used throughout this document. *Section 7* will cite any and all references used throughout this document.

2 Design Overview

This section will go through the implementation details related to the GSMS software and how each software component operates. The diagram detailing the object architecture will be shown, followed by a specific description of each component and the implementation functionality, giving a broad idea of how each component plays a role within the GSMS.

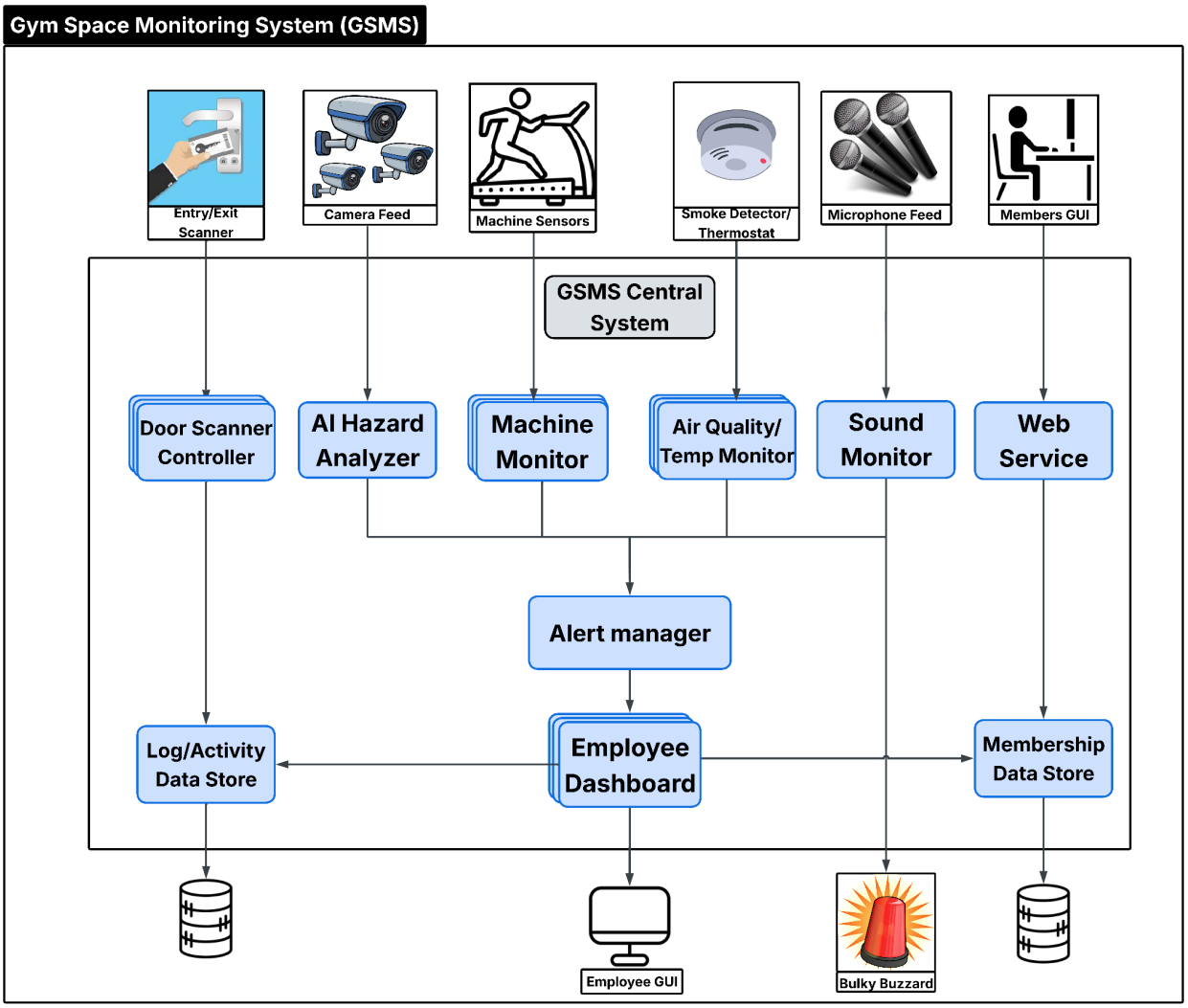

Figure 1: Gym Space Monitoring System Architecture

2.1 Design Explanation

This section describes the architectural design of the Gym Space Monitoring System (GSMS). The system follows a centralized, event-driven architecture in which distributed sensors and monitoring components generate data that is processed by a central computing system.

The following components describe the major functional units of the GSMS and their roles within this architecture:

- **Alert Manager**
  The Alert Manager serves as the central coordination component for event handling within the GSMS. It receives processed signals from monitoring components such as the AI Hazard Analyzer, Machine Monitor, Sound Monitor, and environmental sensors, and evaluates these inputs to determine whether an alert should be

generated. Based on predefined rules, the Alert Manager classifies events into informational, warning, or critical alerts.

Once classified, the Alert Manager records the alert in the Log/Activity Store for auditing and analysis, and dispatches alert notifications to the Employee Dashboard in real time. In cases requiring physical intervention, such as excessive noise, the Alert Manager may also trigger connected output devices (e.g., the Bulky Buzzer). This component ensures consistent alert handling, centralized decision-making, and timely communication across the system.

- **Door Scanner Controller**
  The Door Scanner Controller manages all interactions with the gym's entry and exit scanning system. Upon receiving a scan event, the controller validates the user's membership status by querying the Membership Data Store. If the membership is valid, it authorizes access and records the event for occupancy tracking and historical analysis.

  In addition to access control, the Door Scanner Controller contributes to system-wide occupancy monitoring by updating usage records in the Log/Activity Data Store. It may also forward relevant updates to the Alert Manager if irregular conditions are detected (e.g., invalid access attempts). This component ensures secure, reliable, and trackable entry and exit operations.

- **Employee Dashboard**
  The Employee Dashboard functions as the primary user interface for staff interaction with the GSMS. It provides real-time visibility into system alerts, environmental conditions, occupancy levels, and equipment status. Alerts generated by the Alert Manager are displayed with relevant details such as type, severity, and location, allowing staff to quickly assess and respond to situations.

  In addition to alert monitoring, the dashboard presents historical data retrieved from the Log/Activity Data Store, including usage trends and system logs. The interface supports user actions such as acknowledging alerts, marking issues as resolved, and reviewing system activity. The Employee Dashboard serves as the central operational view of the system, enabling efficient monitoring and response.

- **Web Service**
  The Web Service acts as the backend interface for all member-facing and external application interactions. It processes incoming requests from the Member Web/Mobile Application, including membership management, class registration, and occupancy inquiries.

  To fulfill these requests, the Web Service communicates with the Membership Data Store and Log/Activity Data Store, retrieving or updating relevant data as needed. It then formats and returns responses to the client application. This component ensures secure, structured, and scalable communication between external users and

the GSMS, while maintaining separation between internal system logic and external interfaces.

- **AI Hazard Analyzer**
  The AI Hazard Analyzer is responsible for processing camera data streams to detect potential safety risks within the gym environment. Using computer vision techniques, it analyzes visual input to identify predefined hazard conditions such as falls, overcrowding, improper equipment usage, and obstructed walkways.

  When a potential hazard is detected, the component generates a structured event and forwards it to the Alert Manager for classification and further action. The AI Hazard Analyzer does not directly trigger alerts, but instead provides probabilistic assessments that are validated and managed by downstream components. This separation improves reliability and reduces the impact of false positives.

- **Sound Monitor**
  The Sound Monitor analyzes audio input from the gym environment to detect excessive noise levels or disruptive behavior. When sound levels exceed predefined thresholds, the component generates an event indicating a potential disturbance.

  This event is forwarded to the Alert Manager, which determines the appropriate response. In certain cases, this may result in the activation of the Bulky Buzzer to notify gym members. The Sound Monitor supports maintaining a safe and comfortable environment by identifying and responding to noise-related disruptions.

- **Air Quality/Temp Monitor**
  The Air Quality/Temp Monitor processes environmental sensor data related to temperature, air quality, and hazardous conditions such as smoke or carbon monoxide levels. It continuously evaluates sensor readings against predefined safety thresholds to identify abnormal or unsafe conditions.

  When a threshold is exceeded, the component generates an environmental hazard event and sends it to the Alert Manager for classification and response. This component ensures that environmental safety conditions are monitored in real time, particularly in areas where camera-based monitoring is not permitted.

- **Machine Monitor**
  The Machine Monitor tracks data from instrumented gym equipment to monitor usage patterns, operational status, and potential malfunctions. It collects signals such as activity duration, frequency of use, and equipment state changes, and stores this information in the Log/Activity Data Store for analysis.

  If abnormal behavior or a malfunction is detected, the Machine Monitor generates an event and forwards it to the Alert Manager for alert generation. This component

supports both real-time incident detection and long-term maintenance planning by providing detailed equipment usage data.

3 Component Specification

This section specifies the components and their interactions within the GSMS. It is here where we go into as much detail as possible about how the component's software communicates with other components, and what type of data is getting passed all across the GSMS. This is communicated through descriptions along with specific public functions which fulfills in one way or another the goal of that component.

3.1 Alert Manager

The following functions define the internal behavior of the Alert Manager, outlining how incoming events are processed, alerts are generated, and their lifecycle is managed from creation to resolution.

- **void processMessage(Message msg)**
  Receives and interprets incoming messages from system components, extracting relevant data and converting it into a standardized event. Determines whether the event requires alert generation.

- **Alert createAlert(Event event)**
  Creates a structured alert from the given event, assigning attributes such as type, severity, timestamp, and location. Returns the alert for further processing and handling.

- **void sendMessage(Message msg)**
  Sends messages from the Alert Manager to other system components, such as the employee dashboards. Ensures timely delivery of alert notifications and control signals.

- **void writeToLogs(Alert alert)**
  Records the alert and its associated data into the Log/Activity Data Store. Supports auditing, tracking, and historical analysis of system activity.

- **boolean checkAlertStatus(Alert alert)**
  Checks the current state of the alert to determine if it is active, acknowledged, or resolved. Returns whether further action is still required.

- **void waitForAlertAcknowledged(Alert alert)**
  Monitor the alert until it is acknowledged by a user through the dashboard. Ensures that all actionable alerts receive attention before progressing.

- **void closeAlert(Alert alert)**
  Marks the alert as resolved and updates its status in the system. Ensures the alert is removed from active views and properly finalized in the logs.

3.2 Door Scanner Controller

The Door Scanner Controller handles member entry and exit through the gym's identification scanners. This component receives QR scan input from the physical scanner, validates the member's credentials using the Membership Data Store, determines access permissions based on scanner location and context, and controls the physical door lock state. Additionally, the component updates system occupancy and activity statistics and generates messages for logging and alert handling. This ensures secure, context-aware access control while maintaining accurate tracking of gym usage.

- **scanData recieveScanData():**
  Receives raw QR scan input from the physical identification scanner and returns a ScanData object containing member ID, scanner ID, and timestamp.

- **memberInfo requestMemberValidation(String memberID):**
  Sends a request to the Membership Data Store to validate the scanned member ID and returns a MemberInfo object containing membership status and authorization details.

- **doorType identifyDoorType(int scannerID):**
  Determines the type and location of the scanner (i.e., entrance, exit, class) and returns a DoorType.

- **boolean proccessMembershipValidation(MemberInfo memberInfo):**
  Evaluates whether the member has an active and valid membership and returns true if valid, otherwise false.

- **boolean processAccessDecision(MemberInfo memberInfo, DoorType doorType, boolean classValid):**
  Determines whether access should be granted based on membership validity, door type, and class registration, and returns true if access is approved.

- **boolean checkClassRegistration(String memberID, int classID):**
  Verifies whether the member is registered for the current or upcoming class and returns true if authorized.

- **doorState unlockDoor(boolean accessGranted):**
  Controls the physical door mechanism. If access is granted, sends a signal to unlock the door and returns the updated DoorState; otherwise, keeps the door locked.

- **void updateEntryStats(String memberID, int scannerID, DateTime timestamp):**

Updates system data including occupancy count, entry/exit logs, and activity tracking based on successful access events.

- **void sendToAlertManager(Message message):**
  Sends constructed messages to the Alert Manager for logging and monitoring.

3.3 Employee Dashboard

The Employee Dashboard is the hub for staff to receive and resolve the various alerts and messages which are passed around across the entire system. There are two versions within the dashboard. The standard staff dashboard as well as the manager dashboard which requires special manager credentials to access. The manager dashboard is sent messages and alerts which normal employees receive, along with accessing system logs from the log store, and gym activity data. Authentication is required as only an employee should have access to this information. Additionally alerts can be marked resolved or investigated through the dashboard, with certain resolutions and actions requiring a manager override, where a manager pin is prompted in order to do a certain action related to critical alerts.

- **userRole validateCredentials(String employeeID, String password)**
  Used for user sign in, this function serves as the primary authentication gateway for the system. It passes the provided credentials to the membership data store to verify the employee's identity and their role as well. This is to check if the user has access to the manager dashboard or not.

- **List<Message> fetchAlerts()**
  Queries the alert dispatcher to retrieve a list of all currently active and unresolved alerts. Populates the dashboard with the data retrieved.

- **boolean processManagerOverride(String alertID, String managerPin)**
  Verifies manager identity in order to acknowledge a critical alert. The dashboard will continue to do the action required, but does not cache the manager override permissions if they happen again, unless the account role is a manager.

- **String getGymStatistics(String date)**
  Queries the data store to retrieve gym statistics for a certain date. Statistics include machine usage, traffic, as well as class registrations and memberships. Only accessible to the manager.

- **String getSysLogs(String date)**
  Queries the data store to retrieve system logs. Mainly used for maintenance and investigations, and only accessible to the manager.

3.4 Web Service

The Web Service is the front-facing interface for members to not only sign up for a membership, but also to sign up for classes, and look at activity reports of the gym so that they can plan their visits around high traffic times if they wish. The web service has a

domain which is accessible through the internet, so any device can access it. The internet then serves the website via the web service, which has access to the datastore of the system so that it can properly serve accurate data to customers online. The web service also writes to the data store for membership and class sign-ups.

- **pageInfo getPage():**
  Called on server side page load, gets all the necessary information for the web page to display.

- **boolean validateMember(String memberInfo):**
  Basic login query, checks if the user is a valid member using the membership store, and if not will prompt them to register to be a member.

- **void classRegister(String memberID, String classInfo):**
  Write a request to the membership data store to register for a class. The validation for the class availability and valid member information is done client side.

- **void registerMember(String memberID):**
  Write a request to add a new valid member, validation for correct member info is done on the client side.

- **ActivityLog getActivityLog(String date):**
  Retrieves the traffic information for a particular day, the website averages this information across several days to get up to date information to display to the client.

- **String getClassSchedules(String date):**
  Retrieves the schedule for classes available on a certain date, along with availability and descriptions.

- **MemberInfo getMemberInfo(String memberID)**
  Retrieves any and all data related to a specific member, including a schema to produce a valid QR code for scanning into the gym, and class which that member has signed up for.

3.5 AI Hazard Analyzer

The AI Hazard Analyzer handles camera-derived image data from multiple cameras in different monitored gym zones and checks it for hazard-related conditions relevant to the GSMS. This component prepares image data for an external AI API call, receives the structured result back, and converts that result into standardized system messages for the Alert Manager. In other words, it acts as the layer between raw camera input and the rest of the system's alert flow.

- **FrameAnalysisResult analyzeFrame(byte[] imageBytes, String mimeType, int zoneID):**
  Uses image bytes, MIME type, and zone identifier, submits the frame to the external

AI API, and returns a FrameAnalysisResult containing hazard and occupancy findings.

- **Message buildHazardMessage(FrameAnalysisResult result, int zoneID, DateTime timestamp):**
  Uses the frame analysis result, zone identifier, and timestamp, to return a Message.

- **Message buildOccupancyMessage(FrameAnalysisResult result, int zoneID, DateTime timestamp):**
  Receives the frame analysis result, zone identifier, and timestamp, to return a Message.

- **void sendToAlertManager(Message message):**
  Sends a message to the Alert Manager.

3.6 Sound Monitor

The Sound Monitor handles ambient gym noise from multiple devices, and determines when the Bulky Buzzer should be activated. This component continuously samples sound input, measures the current sound level, compares it against a configured threshold, and updates the Bulky Buzzer state when disruptive noise conditions are detected. For fast response, the threshold-based trigger is handled locally. Recently buffered audio may also be prepared and sent to an external AI API for secondary sound classification. That returned result is then converted into a standardized system message for the Alert Manager.

- **double getSoundLevel():**
  Returns the current normalized sound level detected by the monitoring device.

- **boolean noiseThresholdExceeded(double soundLevel):**
  Uses the current sound level and returns true if the configured Bulky Buzzer threshold has been exceeded.

- **BulkyBuzzerState updateBuzzerState():**
  Returns the current Bulky Buzzer state, including whether it is armed, whether the alarm is active, and the most recent trigger source.

- **AudioAnalysisResult requestSoundAnalysis(byte[] audioBytes, String mimeType):**
  Uses buffered audio bytes and MIME type, submits the audio clip to the external AI API, and returns an AudioAnalysisResult.

- **Message buildSoundMessage(BulkyBuzzerState buzzerState, AudioAnalysisResult audioResult, DateTime timestamp):**
  Uses the current buzzer state, optional audio analysis result, and timestamp, to return a Message.

- **void sendToAlertManager(Message message):**
  Sends a message to the Alert Manager.

3.7 Air Quality/Temp Monitor

The Air Quality/Temp Monitor handles environmental conditions in the gym and determines when a safety alert needs to be sent to the Alert Manager. This component samples air quality and temperature data to determine whether smoke, carbon monoxide, or unsafe temperature levels are present. If one of those bounds is exceeded, it builds the appropriate message and sends it to the Alert Manager for processing.

- **double getTemperature():**
  Returns the current temperature reading from the monitoring device.

- **AirQualityData getAirQuality():**
  Returns the current air quality readings, including values relevant to smoke and carbon monoxide detection.

- **boolean hazardThresholdExceeded(AirQualityData airData, double temperature):**
  Receives the current air quality data and temperature reading and returns true if a hazardous environmental condition has been detected.

- **Message buildEnvironmentalMessage(AirQualityData airData, double temperature, int zoneID, DateTime timestamp):**
  Receives the current air data, temperature, zone identifier, and timestamp, to return a Message.

- **void sendToAlertManager(Message message):**
  Sends a message to the Alert Manager.

3.8 Machine Monitor

The Machine Monitor is present on each supported machine and handles usage tracking and machine error reporting for the GSMS. This component monitors usage time, activity state, and machine-specific faults, then converts those results into messages for the Alert Manager. If an error is detected, the component sends an alert-related message. If the data is only related to usage, it sends a non-alert message for logging, tracking, and later analytics.

- **UsageData getUsageData():**
  Returns the current usage information for the machine.

- **MachineError getMachineErrorState():**
  Returns the current machine error or fault code.

- **Message buildMachineAlertMessage(MachineError error, int machineID, DateTime time):**

Uses the machine error, machine ID, and time to build a Message.

- **Message buildUsageMessage(UsageData usageData, int MachineID, DateTime time)**
  Uses the usage data, machine ID and time to return a Message.

- **void sendToAlertManager(Message message):**
  Sends a message to the Alert Manager.

4 Use Cases

The following are definitions of flow between gym members and/or staff with the GSMS, and the expected behavior of the system and how it should react to these interactions. With the focus on the user, little of the GSMS internals are laid out, instead emphasising the expected outputs.

Use Case 1: Sign in to the Gym
Actor: User
Goal: Enter the gym facility
Preconditions:
- User must have an active membership
- User possesses a valid QR code (either physical or digital)
- Entry scanner is operational

Trigger: User scans QR code at entrance
Primary Component: Door Scanner Controller
Scenario:
1. User approaches the gym entrance
2. User scans their membership QR code
3. The Door Scanner Controller reads the scanned QR code and validates the membership information
   - receiveScanData()
   - requestMemberValidation(String memberID)
   - identifyDoorType(int scannerID)
   - proccessMembershipValidation(MemberInfo memberInfo)
   - processAccessDecision(MemberInfo memberInfo, DoorType doorType, boolean classValid)
4. The system updates entry and occupancy records using the Door Scanner Controller
   - updateEntryStats(String memberID, int scannerID, DateTime timestamp)
5. If the membership is valid, the Door Scanner Controller sends the unlock signal to the entrance door
   - unlockDoor(boolean accessGranted)
6. User sees the door unlock
7. User enters the gym

Use Case 2: Responding to a Fall
Actor: User, Staff, Manager
Goal: Staff and Manager respond to falls efficiently
Preconditions:
- The cameras are on and collecting data
- The manager and staff are effectively receiving alerts

Trigger: Camera's AI detects state change for longer than 10 seconds.
Primary Components: AI Hazard Analyzer, Alert Manager, Employee Dashboard
Scenario:
1. User falls in the gym
2. User is hurt by the fall and stays down
3. The AI Hazard Analyzer processes the camera frame and identifies a possible fall event
    - analyzeFrame(byte[] imageBytes, String mimeType, int zoneID)
4. The AI Hazard Analyzer builds a hazard message and sends it to the alert Manager
    - buildHazardMessage(FrameAnalysisResult result, int zoneID, DateTime timestamp)
    - sendToAlertManager(Message message)
5. The Alert Manager classifies the event as a critical alert and forwards it to the Employee Dashboard
    - processMessage(Message msg)
    - createAlert(Event event)
    - writeToLogs(Alert alert)
    - sendMessage(Message msg)
6. Staff and manager receives a critical alert
    - fetchAlerts()
7. Staff or manager acknowledge the alert through the Employee Dashboard
    - checkAlertStatus(Alert alert)
    - waitForAlertAcknowledged(Alert alert)
8. Staff or manager assess the user's condition
9. Staff or manager provides assistance or first aid
10. Staff or manager call emergency services if necessary
11. The Employee Dashboard updates the alert status and the Alert Manager records the alert as resolved
    - closeAlert(Alert alert)

Use Case 3: Dropping a Heavy Free Weight
Actor: User
Goal: Reduce excessive noise in the gym by alerting users to remain below a noise level
Preconditions:
- Sound monitoring system is active in the cameras
- Bulky Buzzer is operational

Trigger: User drops a heavy weight making a loud sound
Primary Component: Sound Monitor
Scenario:

1. User drops a heavy weight on the ground during lifting, making a loud noise
2. The Sound Monitor samples the current sound level and determines that the configured threshold has been exceeded
    - getSoundLevel()
    - noiseThresholdExceeded(double soundLevel)
3. The Sound Monitor updates the Bulky Buzzer state and activates the buzzer
    - updateBuzzerState()
4. User hears the Bulky Buzzer sound off
5. The system logs the sound-related event
    - buildSoundMessage(BulkyBuzzerState buzzerState, AudioAnalysisResult audioResult, DateTime timestamp)
    - sendToAlertManager(Message message)
6. User realizes they were too loud
7. User uses weights more consciously

Use Case 4: Responding to a Cardio Machine Malfunction
Actor: User, Staff, Manager
Goal: Resolve a cardio machine equipment malfunction
Preconditions:
- Cardio machine sensors are operational
- Staff and managers are receiving alerts

Trigger: Manager receives a malfunction flag from the system
Primary Components: Machine Monitor, Alert Manager, Employee Dashboard
Scenario:
1. The cardio machine begins operating abnormally
2. The Machine Monitor detects a fault or abnormal condition during operation
    - getMachineErrorState()
3. The Machine Monitor generates a fault message and sends it to the Alert Manager
    - buildMachineAlertMessage(MachineError error, int machineID, DateTime time)
    - sendToAlertManager(Message message)
4. The Alert Manager forwards the malfunction alert to the Employee Dashboard
    - processMessage(Message msg)
    - createAlert(Event event)
    - writeToLogs(Alert alert)
    - sendMessage(Message msg)
5. Manager receives malfunction alert
    - fetchAlerts()
6. Manager acknowledges the alert
    - waitForAlertAcknowledged(Alert alert)
7. Staff goes to the machine
8. Staff inspects the issue
9. Staff disables or repairs the machine
10. Manager marks the alert as resolved or calls for new parts
    - checkAlertStatus(Alert alert)

11. The system records the maintenance event and updates the machine status through the Alert Manager and dashboard
    - closeAlert(Alert alert)
12. Manager clears alert and enables machine once it is fixed

Use Case 5: Staff/Manager Acknowledges and Resolves Alert
Actor: Staff, Manager
Goal: Process and resolve system alerts
Preconditions:
- System is actively monitoring and generating alerts
- Staff and Manager has access to central or manager computer
- Alert notification system is operational

Trigger: Staff or Manager receives an alert from the system
Primary Components: Alert Manager, Employee Dashboard
Scenario:
1. Staff or Manager receives alert notification
    - fetchAlerts()
2. The Employee Dashboard displays the alert details received from the Alert Manager
    - fetchAlerts()
3. Staff or Manager opens the alert on the dashboard
4. Staff or Manager acknowledges the alert
5. The Employee Dashboard sends the acknowledgment back to the Alert Manager, which updates the alert lifecycle state
    - waitForAlertAcknowledged(Alert alert)
    - checkAlertStatus(Alert alert)
6. Staff or Manager reviews alert details such as location, type, and severity
7. Staff or Manager goes to the affected location if needed
8. Staff or Manager resolves the issue
9. Staff or Manager marks the alert as resolved in the system
    - closeAlert(Alert alert)
10. The Alert Manager logs the completed alert and returns the system to normal monitoring state
    - writeToLogs(Alert alert)

Use Case 6: Manager Reviews Equipment Usage
Actor: Manager
Goal: Make maintenance or purchasing decisions
Preconditions:
- System is collecting equipment usage data
- Manager has access to manager dashboard
- Data storage and reporting system is operational

Trigger: Manager opens dashboard to review equipment data
Primary Components: Employee Dashboard, Machine Monitor
Scenario:

1. Manager logs into dashboard
    - validateCredentials(String employeeID, String password)
2. The Employee Dashboard requests equipment usage information collected by the Machine Monitor
    - getGymStatistics(String date)
3. The system retrieves stored usage frequency and duration data and displays it on the dashboard
    - getUsageData()
    - buildUsageMessage(UsageData usageData, int MachineID, DateTime time)
    - sendToAlertManager(Message message)
4. Manager selects equipment usage reports
    - getGymStatistics(String date)
5. Manager identifies heavily used or underused machines
6. Manager receives maintenance trends
7. Manager decides to schedule maintenance or purchase equipment
8. Manager records or exports report if needed

Use Case 7: User Signs Up for a Class
Actor: User
Goal: Register for a gym class
Preconditions:

- User has an active membership
- User has access to web app or mobile app
- Class schedule is available

Trigger: User selects a class from the schedule
Primary Component: Web Service
Scenario:

1. User logs into web or mobile app
    - validateMember(String memberInfo)
2. The Web Service verifies the user session and loads available class information
    - getPage()
    - getClassSchedules(String date)
3. User views available classes
4. User selects a desired class
5. The Web Service checks class availability and submits the registration request
    - classRegister(String memberID, String classInfo)
6. If space is available, the Web Service updates the class roster and stores the registration
    - classRegister(String memberID, String classInfo)
7. User is registered
8. The Web Service returns a confirmation message to the user interface
    - getMemberInfo(String memberID)
9. User receives confirmation

Use Case 8: User Checks Gym Occupancy
Actor: User
Goal: Plan visit based on crowd levels
Preconditions:
- System is collecting occupancy data
- User has access to web app or mobile app

Trigger: User opens occupancy feature
Primary Components: Web Service
Scenario:
1. User logs into app
    - validateMember(String memberInfo)
2. The Web Service receives the occupancy request from the user
    - getPage()
3. The Web Service retrieves the most recent occupancy information maintained by the system
    - getActivityLog(String date)
4. User selects occupancy view
5. The system returns the current crowd level information to the user through the Web Service
    - getActivityLog(String date)
6. User reviews crowd levels
7. User decides when to visit gym

Use Case 9: Staff/Manager Responds to Environmental Hazard
Actor: Staff, Manager
Goal: Ensure safety during hazardous conditions
Preconditions:
- Environmental sensors (e.g., smoke, temperature, etc.) are active
- System alert system is operational
- Staff and Manager are receiving alerts

Trigger: System detects environmental hazards (e.g., smoke, high temperature, etc.)
Primary Components: Air Quality/Temp Monitor, Alert Manager, Employee Dashboard
Scenario:
1. Unsafe environmental conditions occur, such as smoke or high temperature
2. The Air Quality/Temp Monitor detects that a configured safety threshold has been exceeded
    - getTemperature()
    - getAirQuality()
    - hazardThresholdExceeded(AirQualityData airData, double temperature)
3. The Air Quality/Temp Monitor generates a hazard message and sends it to the Alert Manager
    - buildEnvironmentalMessage(AirQualityData airData, double temperature, int zoneID, DateTime timestamp)
    - sendToAlertManager(Message message)
4. The Alert Manager classifies the event and dispatches the alert to the Employee Dashboard

    - processMessage(Message msg)
    - createAlert(Event event)
    - writeToLogs(Alert alert)
    - sendMessage(Message msg)
5. Staff or Manager receive hazard alert
    - fetchAlerts()
6. Staff or Manager acknowledge alert
    - waitForAlertAcknowledged(Alert alert)
7. Staff or Manager go to affected area
8. Staff or Manager assess severity of hazard
9. Staff or Manager take appropriate action, such as evacuation or equipment shutdown
10. Staff or Manager notify emergency services if required
11. The Employee Dashboard and Alert Manager update the alert status when the situation is resolved
    - closeAlert(Alert alert)
12. Staff or Manager mark alert as resolved once safe

5 Design Constraints

The design, which was laid out, was built with several constraints in mind. These are specifications and limitations set forth by the client, the law, the gym environment, equipment limitations, and the use of AI-based analysis. Within these guidelines, the system will operate in a manner that is reliable, operational, and easy to manage.

5.1 Maintainability

The system must remain maintainable across distributed sensors, cameras, and software components. Because the GSMS operates across multiple zones with many interconnected devices, the system is designed around a centralized architecture with a local server and dashboards. This allows all system activity, errors, and alerts to be monitored from a single location, reducing the complexity of diagnosing failures.

Additionally, the system uses modular components, allowing individual subsystems to be updated or replaced independently. This design decision ensures that future improvements, especially those to AI processing or sensor integrations, can be made without requiring a full system redesign.

5.2 AI and Data Processing Constraints

The system depends on AI-based computer vision analysis to detect hazards such as falls, aggression, overcrowding, obstructed walkways, and unsafe equipment usage. Because AI-based systems are inherently probabilistic, the design must account for imperfect accuracy, processing latency, and contextual limitations.

Due to these constraints, the GSMS is designed with a human-in-the-loop architecture. This means the AI detections from the AI Hazard Analyzer are not treated as final decisions;

instead, they are routed through the Alert Manager, which classifies events and sends them to staff dashboards for validation.

5.3 Environmental Constraints

The system must operate in a gym environment with heat, humidity, and constant motion. Because the gym environment introduces noise and unpredictability, the GSMS design uses multiple complementary data sources rather than depending on a single sensor type. For example, cameras handle fall detection and occupancy, environmental sensors detect air quality and temperature, and machine sensors track usage.

This multiple data source approach improves reliability in cases where one input source is unreliable. Additionally, system thresholds are designed to be configurable, allowing adaptation to different gym conditions.

5.4 Privacy Constraints

The system's camera monitoring must respect privacy, especially in sensitive areas like locker rooms. Due to strict privacy requirements, the GSMS uses a zone-based monitoring design where different areas of the gym are monitored using different technologies.

More specifically, no cameras are deployed in the locker rooms; environmental sensors are used instead to monitor external conditions. Camera-based monitoring is restricted to the public workout areas.

Additionally, the system enforces role-based access control through the employee and manager dashboards, ensuring that only authorized users can view certain system data.

5.5 Equipment and Integration Constraints

The system must not interfere with existing gym equipment and must support both instrumented and non-instrumented equipment. Because modifying gym equipment is impractical and unsafe, the GSMS is designed as a non-invasive monitoring system.

This is accomplished through direct censoring for compatible machines and indirect observation for non-instrumented equipment. Additionally, the system does not control equipment, but instead flags issues through alerts. This separation ensures compatibility across different equipment types and avoids introducing safety risks.

5.6 System Reliability and Recovery

The system must continue operating under partial failure and ensure alerts are not missed. Because the GSMS is responsible for safety monitoring, the system is designed with graceful degradation and alert persistence.

Key design decisions include the Alert Manager lifecycle, which ensures alerts are tracked until resolved, and escalation logic, where alerts are resent if not acknowledged. Additionally, the centralized system ensures that after a restart or failure, the system can

resume monitoring without requiring manual reconfiguration, maintaining continuity in safety monitoring.

6 Definition of Terms

Throughout the document, certain words will be used frequently when describing the GSMS. The following words are defined precisely in how we intend to use them, and what they refer to when it comes to the GSMS in particular.

- **Gym Space Monitoring System (GSMS):** The official title refers to the whole system, which includes all devices in each room, along with the software components.

- **Management:** The overall administrative acts used to manage the gym, including member sign-up and tracking, upholding building and fire codes, member safety, as well as managing the traffic and flow of members within the gym.

- **Central Computer:** The primary processing unit of the GSMS is responsible for aggregating data from all sensors, performing analysis, and coordinating alerts and system responses.

- **Maintenance:** Referring to the upkeep of the gym as it relates to gym equipment. This refers to the replacement and fixing of faulty or broken gym equipment, along with hygienic practices such as cleaning all equipment at regular intervals.

- **Identification Scanner:** A scanner which is placed at the entrance of the gym, where members are provided with an ID card upon sign-up and can then scan in and scan out of the gym using simple 2D barcode scanning.

- **Bulky Buzzer:** A special alarm which is placed near the front of the gym, which triggers if people or a particular member is being too loud during their exercise, thus making other members uncomfortable and unsafe.

- **Environmental Hazards:** Refers to both extreme hazards, such as fires and other emergencies that warrant evacuation, and to small hazards such as tripping on misplaced gym equipment.

- **Camera:** An AI-powered camera that uses computer vision techniques to monitor gym activity and identify predefined conditions. These cameras are designed to detect indicators of unsafe behavior (e.g., improper form or potential falls) and estimate occupancy within their field of view, enabling the system

- **Alerts:** Messages which are sent to various components throughout the GSMS. There are 3 types of alerts, an informational alert is a type of alert which strictly signals a longing event. A warning alert which is both logged and displayed on employee

dashboards, prompting staff to investigate. Then finally a critical alert which is the highest priority alert, it is logged and prompts staff to look into and resolve immediately.

- **Log/Activity Data Store:** The datastore where the system both reads and writes log information.

- **QR (Quick Response) Code:** A two-dimensional, machine-readable barcode that stores data. This data will be the member name and id for identification and sign in purposes.

7 Demonstration Semantics

Our demonstration will implement the following components from our architecture to simulate the GSMS in real time. Our demonstration will contain real-time AI video analysis that works to trigger alerts within the system to provide a working proof of concept.

- **AI-Hazard Analyzer**
  The AI-Hazard Analyzer component uses a real-time data stream from a webcam that is plugged into the computer. The camera stream is processed within Java and a single frame is selected once per second. That frame is then compressed and sent to the Google Gemini Live API[1] via an API call. The model is given a context prompt explaining its purpose and what it needs to return with strict rules to follow. Once the frame is analyzed an analysis is received from the model in JSON format. That JSON is then parsed and used by the system. Our demonstration will utilize the AI Hazard Analyzer component to demonstrate monitoring occupancy levels, flagging of aggressive behavior, and detecting trip hazards and objects in walkways. The sound analysis component will be utilized to classify the sounds that trigger the Bulky Buzzer for alert logging.

- **Sound Monitor**
  The Sound Monitor component uses a real-time data stream from a microphone that is plugged into the computer. The microphone stream is first analyzed locally to determine if a certain sound level is exceeded. If the set sound level is exceeded the Bulky Buzzer State is changed and the alarm is sounded. To determine the classification of the sound, we then take a sound byte from around the time of the sound and send it via API call to be analyzed. The model is once again given a context prompt and strict rules to follow. Once the sound byte is analyzed an analysis report is received in JSON format that is utilized by the system. The sound monitor will be utilized to classify the sounds that trigger the Bulky Buzzer for alert logging.

- **Alert Manager**
  The Alert Manager component will be implemented according to the architecture utilizing a messaging system to communicate with components and act as the hub of the system. The messages will be assessed and turned into the appropriate alerts to

be sent to the employee dashboard. A log store will be generated so that alerts can be reviewed and seen over a span of time.

- **Employee Dashboard**
  The employee dashboard will be implemented in JavaFX as a GUI that communicates with the alert manager to display alerts as they come in. It will be implemented as a Manager Dashboard and Standard Employee Dashboard to work with different levels of alerts and show the different information that each level of employee receives. Each will be implemented as an individual GUI so that they can display the required information that differs between the two instances.

- **Door Scanner Controller**
  The door scanner controller will be implemented in Java using a Zebra scanner that can scan QR codes. The scanner functions as keyboard input so types in and enters whatever data it reads from the QR code. This is a simpler version of what would be in place of the full system, but this implementation will take the user info from the QR code and show a member scanned into the gym.

8 References

[1] "Gemini Live API Overview | Gemini API | Google AI for Developers." *Google*, Google, ai.google.dev/gemini-api/docs/live-api. Accessed 21 Apr. 2026.

[2] Lucid Software Inc. "Lucidchart." Lucid Software, www.lucidchart.com.

[3] *Fire Code*, www.cabq.gov/fire/documents/city-of-albuquerque-fire-code.pdf. Accessed 28 Mar. 2026.

[4] "NEW MEXICO COMMERCIAL BUILDING CODE." *14.7.2 NMAC*, www.srca.nm.gov/parts/title14/14.007.0002.html. Accessed 28 Mar. 2026.